

Multi-threading & Concurrency — Senior Developer Level

JavaScript is often described as **single-threaded**, but that statement is incomplete.

The JavaScript code executing on a particular thread runs sequentially, but modern JavaScript environments can provide:

- Multiple threads
- Worker threads
- Web Workers
- Shared memory
- Message passing
- Background workers
- OS-level processes
- Thread pools
- Parallel CPU execution

The important question is not:

"Is JavaScript single-threaded?"

A better question is:

"Which execution environment is running the JavaScript, and how does it achieve concurrency or parallelism?"

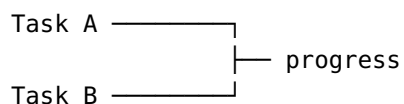
1. Concurrency vs Parallelism

Before workers, understand these two concepts.

Concurrency

Concurrency means multiple tasks can make progress during overlapping periods.

Example:



They don't necessarily execute at the exact same time.

Parallelism

Parallelism means tasks actually execute simultaneously on different CPU cores.

CPU Core 1 → Task A
CPU Core 2 → Task B

So:

Concurrency ≠ Parallelism

A system can be concurrent without executing tasks in parallel.

2. Why JavaScript Needs Workers

Consider:

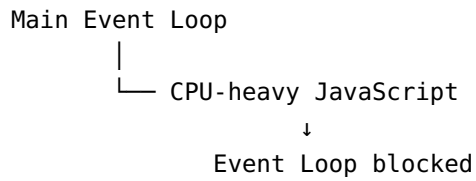
```
function calculate() {  
  for (let i = 0; i < 10_000_000_000; i++) {  
    // expensive calculation  
  }  
}
```

If this runs on the browser's main thread:

```
Browser Main Thread  
  |  
  ├── UI  
  ├── click events  
  ├── rendering  
  ├── JavaScript  
  └── expensive calculation  
      ↓  
    BLOCKED
```

The UI can become frozen.

In Node.js:



Workers allow CPU-heavy JavaScript to execute elsewhere.

3. I/O-bound vs CPU-bound

This distinction is extremely important.

I/O-bound work

Examples:

- Database request
- Network request
- File I/O
- HTTP request

Node.js is already very good at handling many I/O operations concurrently.

```
graph LR; A[Request A] --> I1[I/O] --> R1[result]; B[Request B] --> I2[I/O] --> R2[result]; C[Request C] --> I3[I/O] --> R3[result];
```

You usually don't need a worker just because an operation is asynchronous.

CPU-bound work

Examples:

- Image processing
- Video encoding
- Hashing
- Compression
- Large mathematical calculations
- ML inference

- Parsing huge datasets
- Cryptographic calculations

These can consume CPU for a long time.

CPU-heavy task
↓
Main thread blocked

Workers are especially useful here.

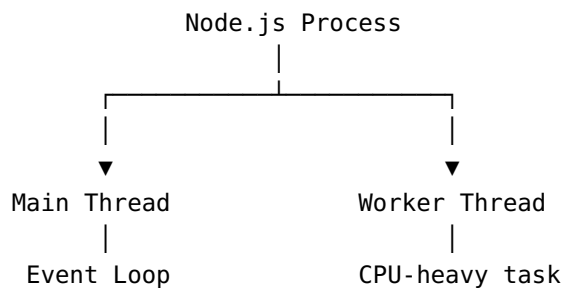
4. Node.js Worker Threads

Node.js provides:

```
import { Worker } from "node:worker_threads";
```

A Worker runs JavaScript in another thread.

Conceptually:



The worker has its own:

- JavaScript execution context
- Event loop
- V8 isolate
- Heap

It does not simply execute inside the same JavaScript execution context as the main thread.

5. Basic Worker Example

Main file:

```
import { Worker } from "node:worker_threads";

const worker = new Worker("./worker.js");

worker.on("message", (result) => {
  console.log("Result:", result);
});

worker.postMessage(10);
```

Worker:

```
import { parentPort } from "node:worker_threads";

parentPort.on("message", (value) => {
  const result = value * value;

  parentPort.postMessage(result);
});
```

Flow:

```
Main Thread
  |
  | postMessage(10)
  ▼
Worker Thread
  |
  | calculate
  ▼
  100
  |
  | postMessage()
  ▼
Main Thread
```

6. Worker Communication

The primary communication mechanism is:

```
worker.postMessage(data);
```

and:

```
parentPort.postMessage(data);
```

This is **message passing**.

Instead of sharing normal JavaScript objects directly:

Main Heap		Worker Heap
Object A	—X—	Object B

you normally send data between the two contexts.

7. Structured Clone

When data is sent using `postMessage()`, JavaScript environments generally use the **structured clone algorithm** for supported values.

Conceptually:

```
Main thread
Object
|
| structured clone
▼
Worker
Cloned Object
```

The worker receives its own representation.

Therefore:

```
const user = {  
  name: "Shariar"  
};  
  
worker.postMessage(user);
```

doesn't mean the worker suddenly has direct access to the original normal object in the main thread's heap.

8. Why Cloning Can Be Expensive

Suppose you send:

500 MB object

through message passing.

Conceptually:

```
graph TD; A[Main memory] -- "clone/serialization work" --> B[Worker memory];
```

That can involve:

- CPU overhead
- Memory overhead
- Latency

This leads to an important optimization:

Transfer ownership when possible instead of cloning large binary data.

That's where transferable objects come in.

9. Worker Threads vs `child_process`

These are related but fundamentally different.

Worker Threads

```
One process
├── Main thread
├── Worker 1
└── Worker 2
```

Workers are threads inside the same process.

Child Processes

```
Process A
└── Node.js process

Process B
└── Node.js process
```

A child process has its own:

- Process memory
- V8 instance
- OS process identity

10. Worker Threads vs `child_process`

Feature	Worker Threads	<code>child_process</code>
Execution unit	Thread	Process
Memory	Can share selected memory	Separate
Startup cost	Usually lower	Usually higher
SharedArrayBuffer	Yes	Not direct across processes

Feature	Worker Threads	<code>child_process</code>
Isolation	Lower	Higher
Crash isolation	Lower	Higher
Best for	CPU parallelism	Process isolation / external programs
Communication	Message passing	IPC
Run another executable	No	Yes

11. When to Use Worker Threads

Use Worker Threads when:

```
Same application
+
CPU-heavy JavaScript
+
Parallel execution
```

Examples:

```
Image resizing
PDF processing
CPU-heavy parsing
Large calculations
Data transformation
```

12. When to Use `child_process`

Use a child process when you need:

- Process isolation
- An external executable
- A separate Node.js process
- Stronger crash isolation

Example:

```
import { spawn } from "node:child_process";

const child = spawn("ffmpeg", [
  "-i",
  "input.mp4",
  "output.mp4"
]);
```

Here Node is running an external program.

That's different from:

```
new Worker("./worker.js");
```

13. Worker Lifecycle

A worker isn't just:

create → execute

Think:

```
Created
  ↓
Started
  ↓
Listening / processing
  ↓
Message exchange
  ↓
Complete
  ↓
Terminate
```

A worker can terminate because:

- Its code finishes
- It calls `process.exit()`
- The parent terminates it
- An uncaught exception occurs

14. Worker Error Handling

Always consider:

```
worker.on("error", (error) => {  
  console.error(error);  
});
```

And:

```
worker.on("exit", (code) => {  
  console.log("Worker exited:", code);  
});
```

Production worker management should consider:

```
message  
error  
exit  
timeout  
cancellation  
retry
```

15. Worker Pool Pattern

Creating one worker per task is usually a bad design.

Imagine:

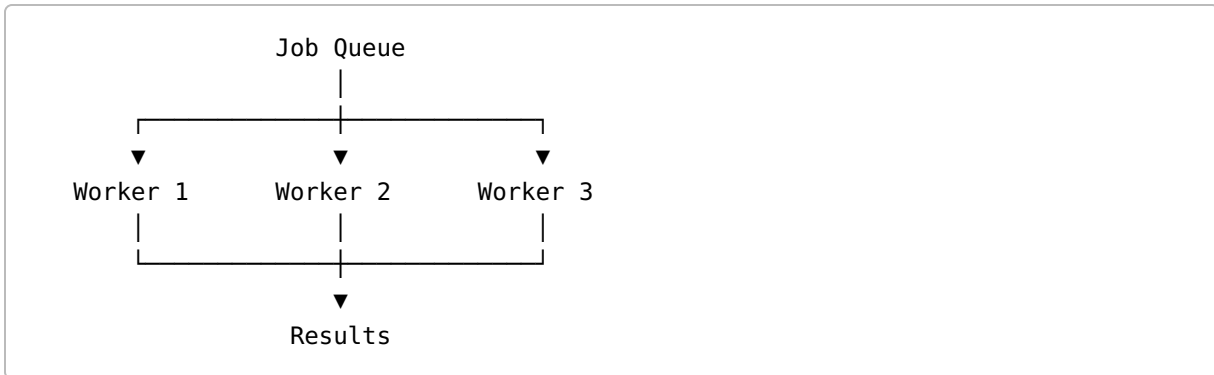
```
1000 jobs  
  ↓  
1000 workers
```

This can create:

- Huge memory usage
- Thread scheduling overhead
- Startup overhead

- CPU contention

Instead, use a **worker pool**.

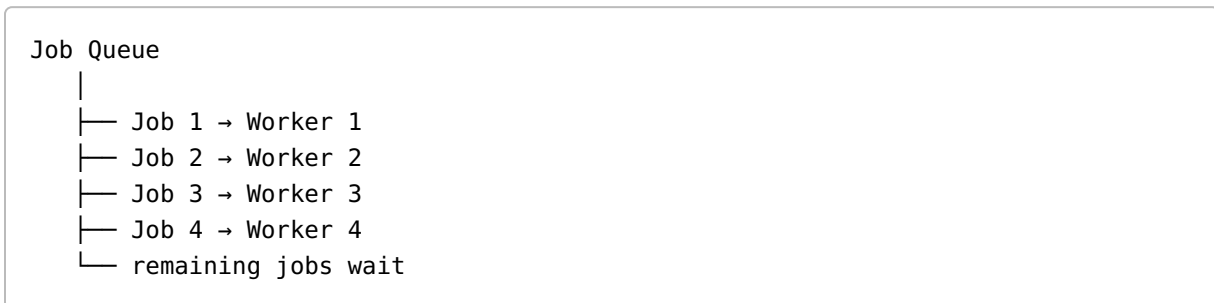


16. Worker Pool Mental Model

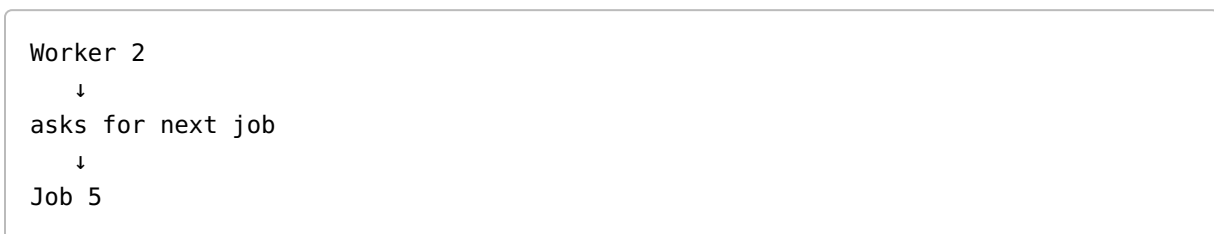
Suppose:

CPU cores = 8

You might maintain a controlled number of workers rather than creating thousands.



When Worker 2 finishes:



This creates controlled parallelism.

17. Worker Pool vs Promise Concurrency

This distinction is important.

A Promise concurrency limiter:

```
10 async HTTP requests  
max 3 at a time
```

does not necessarily create 3 CPU threads.

Workers:

```
Worker 1  
Worker 2  
Worker 3
```

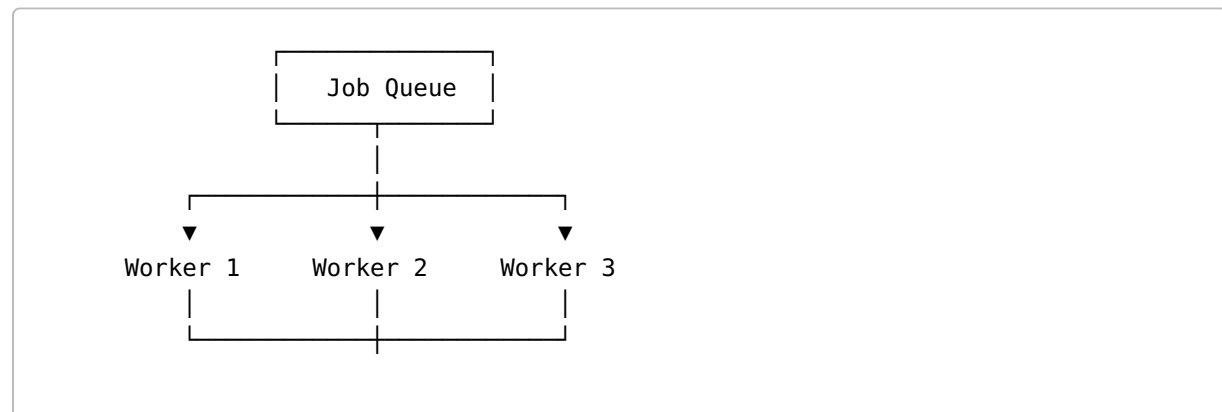
actually provide separate JavaScript execution contexts.

So:

```
Promise concurrency  
    ≠  
Thread parallelism
```

18. Worker Pool Architecture

A robust worker pool can contain:



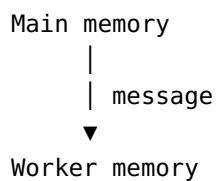
▼
Result Handler

Pool responsibilities:

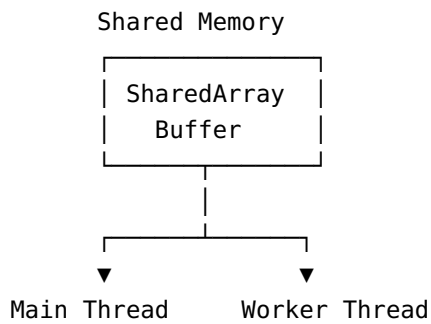
- Create workers
- Assign jobs
- Track busy/idle workers
- Handle worker errors
- Replace crashed workers
- Handle timeouts
- Shut down gracefully

19. SharedArrayBuffer

Normally, worker communication looks like:



With `SharedArrayBuffer`, selected memory can be shared between agents.



This is fundamentally different from ordinary cloned messages.

20. Creating Shared Memory

Example:

```
const shared = new SharedArrayBuffer(1024);
```

You normally create a typed array view over it:

```
const numbers = new Int32Array(shared);
```

Then:

```
numbers[0] = 42;
```

A worker can receive the shared buffer and access the same underlying memory.

21. Why Shared Memory Is Dangerous

Once threads can access the same memory, you can have race conditions.

Imagine:

```
Shared counter = 0
```

Thread A:

```
read 0  
add 1  
write 1
```

Thread B:

```
read 0  
add 1  
write 1
```

Expected:

```
2
```

Actual:

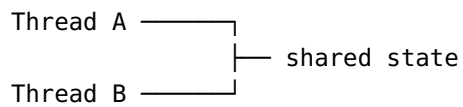
1

This is a **race condition**.

22. Race Condition

A race condition occurs when the result depends on the timing/order of concurrent operations.

Conceptually:



If both manipulate shared state without synchronization:

Unpredictable result

23. Atomics

JavaScript provides:

Atoms

for synchronization and atomic operations on shared typed-array memory.

Example:

```
Atoms.add(numbers, 0, 1);
```

This operation is atomic.

Conceptually:

```
read
+
write
```

is treated as one indivisible operation for the relevant atomic operation.

24. Atomics Example

Without Atomics:

```
numbers[0]++;
```

Conceptually:

```
read
↓
add
↓
write
```

Another thread can interfere.

With:

```
Atoms.add(numbers, 0, 1);
```

the operation is synchronized atomically.

25. Atomics Load / Store

You can also use:

```
Atoms.load(numbers, 0);
```

and:

```
Atoms.store(numbers, 0, 100);
```

These provide atomic access semantics to shared memory.

26. Atomics Wait / Notify

Atoms can also coordinate workers.

Conceptually:

```
Worker A
  |
  | waiting
  ▼
Atoms.wait()
  |
  | notify
  ▲
Atoms.notify()
  |
Worker B
```

This can be used to build synchronization mechanisms.

However, shared-memory synchronization is advanced territory.

Use it only when the performance/architecture actually requires it.

27. Transferable Objects

There are two different concepts:

```
Clone
Transfer
```

Clone

```
Main
|
| copy
▼
Worker
```

Both sides have their own copy.

Transfer

```
Main
|
| ownership transfer
▼
Worker
```

The underlying resource changes ownership instead of being copied in the normal way.

28. ArrayBuffer Transfer

A common transferable object is:

```
ArrayBuffer
```

Example:

```
const buffer = new ArrayBuffer(1024);

worker.postMessage(buffer, [buffer]);
```

The second argument specifies transferable objects.

After transfer, the original side generally can no longer use the transferred `ArrayBuffer` as before; it becomes detached.

29. Why Transfer Matters

Suppose:

100 MB binary data

Clone:

```
graph TD
    Main --> Data[100 MB]
    Main --> Copy[copy → Worker]
    Copy --> Worker[100 MB]
```

Potentially:
200 MB total

Transfer:

```
graph TD
    Main --> Ownership[ownership → Worker]
    Ownership --> Worker[100 MB]
```

This can significantly reduce copying overhead.

30. Zero-Copy Mental Model

"Zero-copy" is best understood as:

Avoiding an additional user-space copy of the underlying transferable memory during ownership transfer.

Conceptually:

Before:

Main

↓
Buffer A

Transfer

Worker
↓
same underlying transferable storage

But don't interpret this as:

"No data movement of any kind ever happens."

The exact implementation details depend on the runtime and object.

31. SharedArrayBuffer vs Transferable

This is a common interview question.

Feature	SharedArrayBuffer	Transferable ArrayBuffer
Shared memory	Yes	No
Ownership	Shared	Transferred
Original remains usable	Yes	No, buffer becomes detached
Synchronization needed	Often	Usually less
Race conditions	Possible	Shared-state race avoided
Use case	Shared state	Efficient one-way data transfer

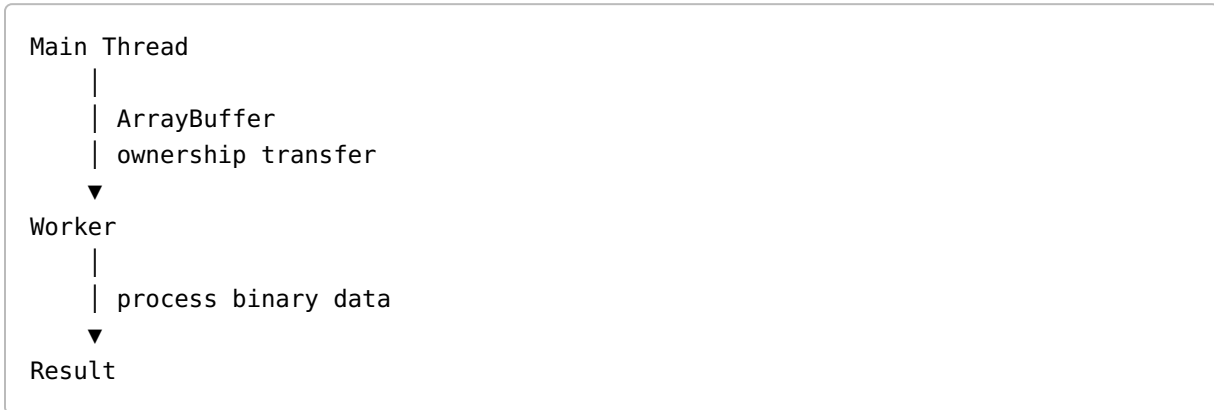
Mental model:

SharedArrayBuffer
↓
"Both can access"

Transferable
↓
"Ownership moves"

32. Worker Threads + Transferable Data

Example architecture:



This is useful for:

- Image data
- Audio buffers
- Binary parsing
- Large typed arrays
- Computational workloads

33. Comlink

Workers often require verbose message-passing code.

Without an abstraction:

```
worker.postMessage({
  type: "calculate",
  payload: data
});

worker.on("message", (result) => {
  ...
});
```

For larger applications, this can become cumbersome.

Comlink provides an RPC-like abstraction over worker communication.

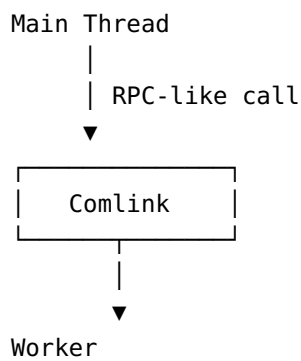
Instead of thinking:

```
postMessage()  
on("message")
```

you can think:

```
await workerProxy.calculate(data)
```

Conceptually:



34. Comlink Mental Model

Comlink makes a remote worker API feel more like a local object.

Conceptually:

```
const workerApi = Comlink.wrap(worker);
```

Then:

```
const result = await workerApi.calculate(10);
```

Behind the scenes:

```
calculate(10)  
↓
```

```
Comlink
  ↓
postMessage()
  ↓
Worker
  ↓
execute
  ↓
postMessage(result)
  ↓
Promise resolves
```

So Comlink is essentially an abstraction over message passing / RPC.

35. Important Comlink Limitation

Comlink does **not** magically make worker memory shared.

The underlying communication still has semantics around:

- Structured cloning
- Transferables
- Proxies
- Serialization
- Worker lifecycle

So:

```
Comlink
  ≠
Shared memory
```

It is primarily an ergonomics/communication abstraction.

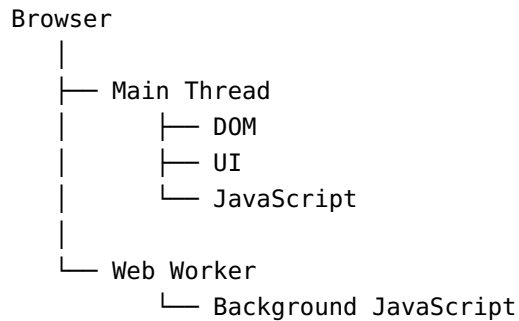
36. Web Workers

Browsers provide:

```
new Worker(...)
```

for background JavaScript execution.

Architecture:



The worker cannot normally manipulate the DOM directly.

37. Why Web Workers Exist

Suppose:

```
const result = expensiveCalculation();
```

on the main thread.

The browser may become:

UI
↓
[A horizontal bar composed of several black squares, representing a frozen or unresponsive UI.]
Frozen

Move calculation to:

Web Worker

and:

Main Thread
↓

UI remains responsive

Worker

↓

expensive calculation

38. Web Worker Communication

Main thread:

```
const worker = new Worker("worker.js");

worker.postMessage({
  type: "calculate",
  value: 100
});
```

Worker:

```
self.onmessage = (event) => {
  const value = event.data.value;

  const result = value * value;

  self.postMessage(result);
};
```

Main receives:

```
worker.onmessage = (event) => {
  console.log(event.data);
};
```

39. Communication Patterns

There isn't just one worker communication pattern.

Important patterns include:

1. Request → Response

```
Main
  ↓ request
Worker
  ↓ result
Main
```

Best for:

- Calculations
 - Parsing
 - One-off jobs
-

2. Fire-and-forget

```
Main
  ↓ command
Worker
```

No immediate response expected.

Useful for:

- Background logging
 - Cache updates
 - Non-critical work
-

3. Streaming messages

```
Main
  ↓
Worker
  ├── progress 10%
  ├── progress 20%
  ├── progress 30%
  └── complete
```

Useful for long-running tasks.

4. Bidirectional communication

Main $\rightleftharpoons$ Worker

Both sides continuously exchange messages.

40. Message Correlation / Request IDs

If many requests are sent to one worker:

```
request 1  
request 2  
request 3
```

responses may arrive in different orders.

Use IDs:

```
worker.postMessage({  
  id: "req-123",  
  type: "calculate",  
  payload: data  
});
```

Worker:

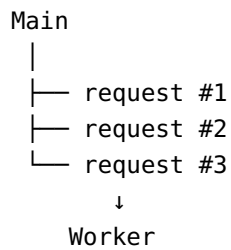
```
parentPort.postMessage({  
  id: "req-123",  
  result  
});
```

Then the main thread can resolve the correct Promise.

This is a very useful production pattern.

41. Worker as RPC Server

You can build:



The worker behaves like a tiny RPC server.

Conceptually:

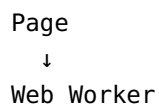
```
function callWorker(type, payload) {  
  return new Promise((resolve, reject) => {  
    // send request with unique ID  
  });  
}
```

This is essentially what abstractions like Comlink help simplify.

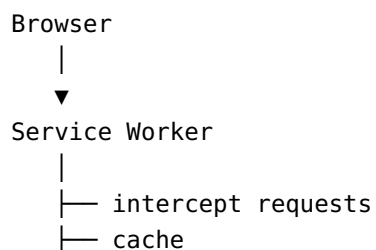
42. Service Workers

A **Service Worker** is different from a normal Web Worker.

Web Worker:



Service Worker:



- └ background functionality
- └ notifications

A Service Worker acts as a programmable network/background layer for a web application.

43. Service Worker Lifecycle

A simplified lifecycle:

```
Register
  ↓
Install
  ↓
Waiting
  ↓
Activate
  ↓
Running
  ↓
Idle / terminated
  ↓
Restarted when needed
```

Important:

A Service Worker is not a permanently running background process.

The browser can stop it when it is idle and start it again when necessary.

44. Service Worker Registration

Main page:

```
navigator.serviceWorker.register("/sw.js");
```

This starts the registration/update lifecycle.

45. Install Event

Inside the Service Worker:

```
self.addEventListener("install", (event) => {  
  console.log("Installing");  
});
```

The install stage is commonly used for preparing resources such as cache entries.

Example concept:

```
self.addEventListener("install", (event) => {  
  event.waitUntil(  
    caches.open("app-cache")  
  );  
});
```

46. Activate Event

```
self.addEventListener("activate", (event) => {  
  console.log("Activated");  
});
```

Activation is commonly used for:

- Cleaning old caches
- Taking control
- Migration logic

Conceptually:

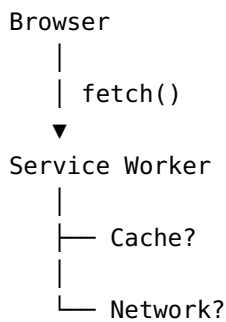
```
Old cache  
  ↓  
cleanup  
  ↓  
New Service Worker
```

47. Fetch Event

Service Workers can intercept requests:

```
self.addEventListener("fetch", (event) => {  
  // inspect request  
});
```

Conceptually:



This is the foundation of many offline-first/PWA architectures.

48. Service Worker vs Web Worker

Very important distinction:

Feature	Web Worker	Service Worker
Main purpose	Background computation	Network/background web functionality
DOM access	No	No
Intercepts fetch	No	Yes
Lifecycle	Usually tied to owner	Browser-managed
Can be terminated when idle	Yes	Yes
Offline caching	Not its primary role	Major use case
Communication	Message passing	Message passing

Mental model:

Web Worker

↓

"Do computation away from UI"

Service Worker

↓

"Act as a background/network layer for the web app"

49. Service Worker Scope

A Service Worker generally controls pages within its scope.

For example:

```
/service-worker.js
```

can control URLs under an appropriate scope depending on registration and server configuration.

This is why Service Worker placement and scope matter.

50. Service Worker Updates

Service Workers have an unusual update lifecycle.

A new version can be:

New Worker

↓

Installing

↓

Waiting

while an older worker is still controlling existing pages.

Then:

```
activate
```

happens according to the lifecycle rules.

This prevents a new Service Worker from abruptly replacing the currently controlling worker in the middle of active page execution.

51. `skipWaiting()` and `clients.claim()`

These APIs can change normal activation/control timing.

Conceptually:

```
skipWaiting()
  ↓
activate sooner

clients.claim()
  ↓
take control of matching clients
```

But these should be used deliberately.

Aggressive activation can create version-mismatch problems between:

```
Old page
+
New Service Worker
```

52. Worker Communication — `MessageChannel`

For more structured communication, browsers provide:

```
MessageChannel
```

It creates two connected ports:

Port 1  Port 2

Example:

```
const channel = new MessageChannel();

worker.postMessage(
  { type: "connect" },
  [channel.port2]
);
```

Then:

```
channel.port1.onmessage = (event) => {
  console.log(event.data);
};
```

This allows a dedicated communication channel.

53. MessagePort

A `MessagePort` represents one endpoint of a message channel.

Conceptually:

```
Main
|
| MessagePort
▼
Worker
```

It can be transferred to another execution context.

This is useful for building more sophisticated communication architectures.

54. Dedicated vs Shared Workers

Browsers also have:

Dedicated Worker

Usually belongs to one page/script context.

```
Page
  ↓
Dedicated Worker
```

Shared Worker

Can potentially be shared by multiple browsing contexts under the appropriate origin.

```
Tab A ┌
Tab B ┤ ──> Shared Worker
Tab C └
```

Shared Workers are less commonly used in modern application architectures, but understanding the distinction is useful.

55. Worker Communication with Transferables

Suppose you're processing an image.

Bad:

```
Main
|
| 100 MB copy
▼
Worker
```

Better:

```
Main
|
| transfer ArrayBuffer
▼
Worker
```

Even better for certain algorithms:

```
SharedArrayBuffer
  ↓
Main ≠ Worker
```

But shared memory requires synchronization.

56. Parallelism Doesn't Mean "More Workers = Faster"

Suppose CPU has:

```
8 cores
```

Creating:

```
100 workers
```

does not mean:

```
100× faster
```

Instead:

```
100 workers
  ↓
CPU contention
  ↓
context switching
  ↓
memory pressure
  ↓
possibly worse performance
```

Worker count should be based on:

- CPU availability
- Workload
- Memory

- Worker startup cost
 - Other application tasks
 - Whether tasks are CPU-bound
-

57. Amdahl's Law

A useful senior-level performance concept:

The maximum speedup of a parallel system is limited by the portion of work that cannot be parallelized.

Suppose:

90% parallel
10% sequential

Even with infinitely many workers:

Maximum speedup $\approx 10\times$

So before creating workers, ask:

How much of this workload is actually parallelizable?

58. Worker Startup Cost

A worker isn't free.

Creating a worker involves runtime setup.

Therefore:

```
Task
↓
create Worker
↓
initialize
↓
```

```
load code
↓
process
↓
terminate
```

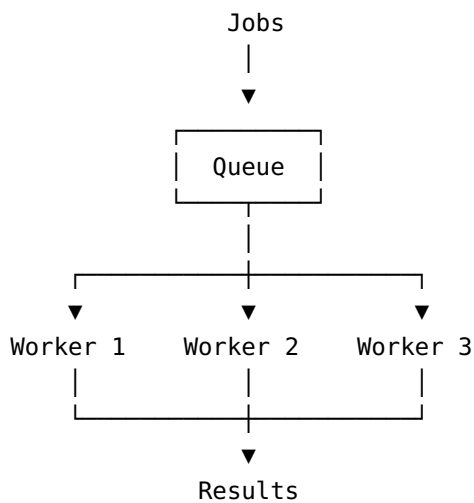
For tiny tasks:

```
worker overhead > actual work
```

A worker pool helps amortize this cost.

59. Worker Pool + Queue

The ideal architecture for many CPU tasks:



This gives you:

```
bounded concurrency
+
resource reuse
+
load control
+
better throughput
```

60. Worker Pool Failure Handling

A production worker pool must answer:

What if a worker crashes?

```
Worker 2
  ↓
crash
```

Pool can:

```
detect exit
  ↓
remove worker
  ↓
requeue unfinished job
  ↓
create replacement worker
```

61. Worker Timeout

Never assume a worker always finishes.

Conceptually:

```
Job
  ↓
Worker
  ↓
timeout
  ↓
terminate worker
  ↓
retry/reject job
```

This protects the application from:

- Infinite loops
- Deadlocks

- Unexpected CPU-heavy work

62. Cancellation

Worker tasks may need cancellation.

A useful architecture:

```
Main
|
| jobId
▼
Worker
|
| running
▼
cancel message
```

The worker can periodically check a cancellation signal/message.

For CPU loops:

```
for (...) {
  if (cancelled) {
    break;
  }

  doWork();
}
```

JavaScript worker execution is cooperative in this sense; simply creating an `AbortSignal` does not magically interrupt arbitrary synchronous CPU code.

63. Shared Memory Requires Synchronization

If you use:

```
SharedArrayBuffer
```

you should immediately think:

Race condition?
Data race?
Atomic operation?
Synchronization?

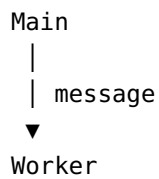
Shared memory is powerful but significantly harder to reason about than message passing.

A useful rule:

Prefer message passing unless shared memory is genuinely necessary.

64. Message Passing vs Shared Memory

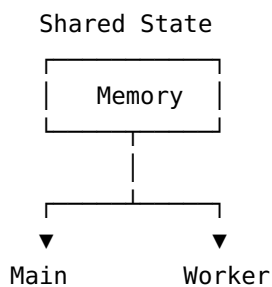
Message Passing



Advantages:

- Easier reasoning
 - Less shared-state complexity
 - Fewer race conditions
-

Shared Memory



Advantages:

- Potentially lower communication overhead
- Useful for high-throughput shared data
- Avoids cloning in certain architectures

Disadvantages:

- Race conditions
- Synchronization complexity
- Harder debugging

65. Immutability Helps Concurrency

Concurrency becomes easier when shared mutable state is minimized.

Instead of:

```
sharedState.count++;
```

prefer architectures where workers receive:

```
Input
↓
Worker
↓
Result
```

This resembles functional programming:

```
Pure input → deterministic computation → output
```

It dramatically reduces synchronization problems.

66. Worker Architecture for Backend

Suppose your Node.js backend receives:

```
POST /process-image
```

Bad:

```
HTTP Request
  ↓
Main Event Loop
  ↓
Huge image processing
  ↓
Event Loop blocked
```

Better:

```
HTTP Request
  ↓
Main Event Loop
  ↓
Job Queue
  ↓
Worker Pool
  ├───┴───┐
  ▼   ▼   ▼
W1   W2   W3
  └───┴───┘
  ↓
Result
  ↓
HTTP Response
```

For long-running jobs, you may go further:

```
HTTP Request
  ↓
enqueue job
  ↓
return jobId
  ↓
background worker pool
  ↓
store result
```

67. Workers Are Not a Replacement for a Job Queue

This is an important backend distinction.

A Worker Pool:

Lives inside your application process

A durable job queue:

Lives as an external system/service

For example:

```
API Server
  ↓
Job Queue
  ↓
Worker Process
  ↓
Database/Object Storage
```

A worker thread alone may not provide:

- Durable jobs
- Cross-process coordination
- Retry persistence
- Distributed processing
- Horizontal scaling

So don't automatically solve every background job problem with Worker Threads.

68. Worker Threads vs External Job Workers

Use Worker Threads when:

Need CPU parallelism
inside same Node process

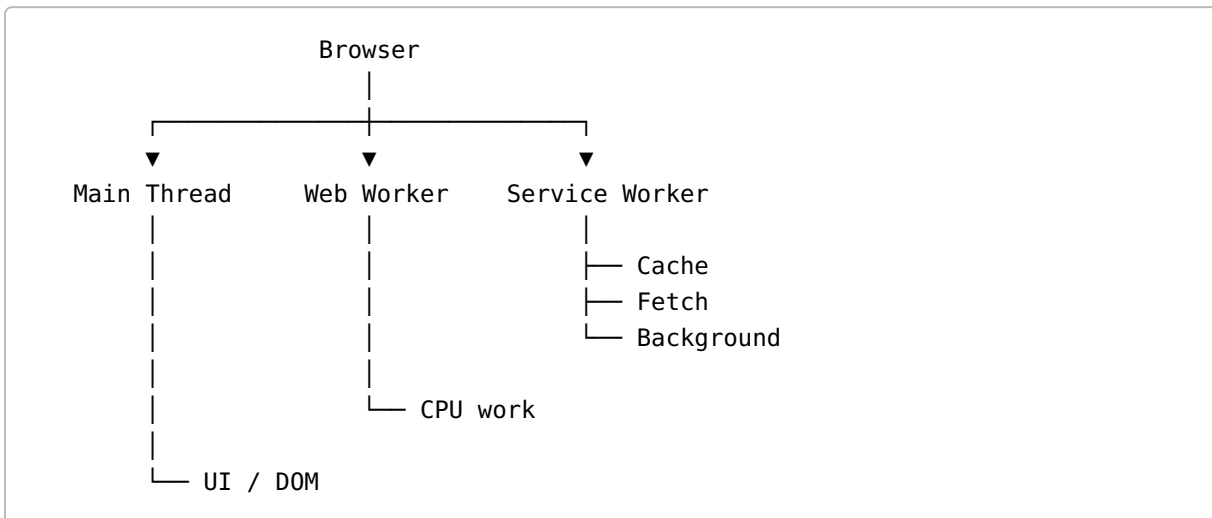
Use separate worker processes/services when:

Need isolation
+
independent scaling
+
durability
+
distributed execution

This distinction becomes very important at scale.

69. Browser Architecture

Modern web applications may have:



Each worker type has a different responsibility.

70. Important Security Boundary

Workers operate in separate execution contexts, but they are not automatically a security boundary equivalent to a separate OS process.

Don't think:

```
Worker = sandbox against all attacks
```

Instead:

```
Worker  
  ↓  
different JS execution context
```

For strong isolation requirements, process/container/service-level isolation may be more appropriate.

71. Common Mistakes

Mistake 1 — Using Workers for I/O

Don't create workers simply because you have:

```
await fetch(...)
```

or:

```
await database.query(...)
```

Asynchronous I/O does not automatically require CPU threads.

Mistake 2 — Creating a worker per tiny task

```
10,000 tiny tasks  
  ↓  
10,000 workers
```

Usually terrible.

Use:

```
Worker Pool  
+  
Job Queue
```

Mistake 3 — Sharing everything

```
SharedArrayBuffer
```

is powerful but adds synchronization complexity.

Prefer message passing where practical.

Mistake 4 — Forgetting transferables

For large binary data:

```
Huge ArrayBuffer
```

blindly cloning it can be expensive.

Consider transfer when ownership can move.

Mistake 5 — Assuming Comlink provides parallelism

Comlink is an RPC-style abstraction.

It doesn't create the worker.

```
Comlink  
↓  
communication abstraction
```

The Worker provides the separate execution context.

Mistake 6 — Confusing Web Worker and Service Worker

Web Worker
→ background computation

Service Worker
→ network/background web application layer

72. Decision Tree

When you have expensive work:

Is it I/O-bound?
|
YES
|
▼
Use async I/O

If:

CPU-bound?
|
YES
|
▼
Can it run in parallel?
|
YES
|
▼
Worker Threads / Web Workers

Then:

Many CPU jobs?
|
YES
|

▼
Worker Pool

If:

Need durable/distributed jobs?
|
YES
|
▼
External Job Queue + Worker Processes/Services

73. Practical Architecture Examples

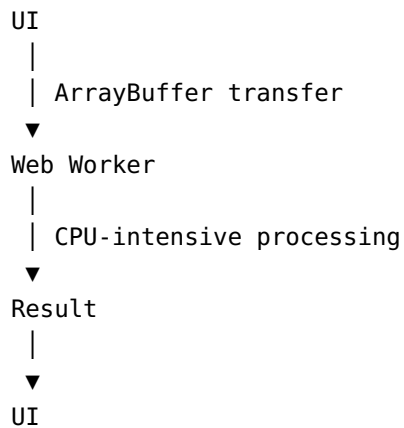
Image Processing

HTTP API
↓
Worker Pool
↓
Image processing
↓
Object Storage

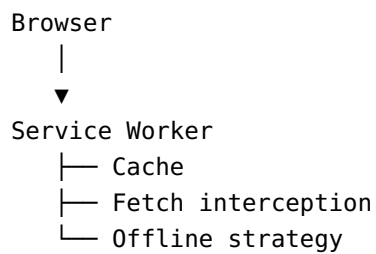
Large Data Parsing

File
↓
Readable Stream
↓
Worker / Worker Pool
↓
Parsing
↓
Database

Browser Video Processing

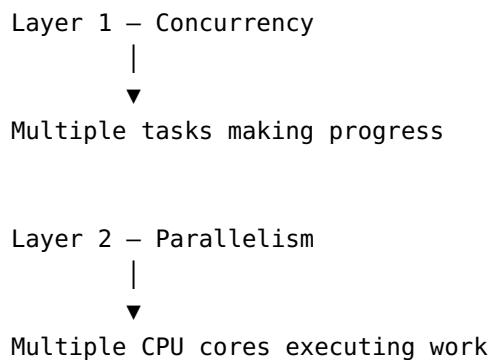


Offline Web App



74. Senior Mental Model

Think about the whole topic in four layers:



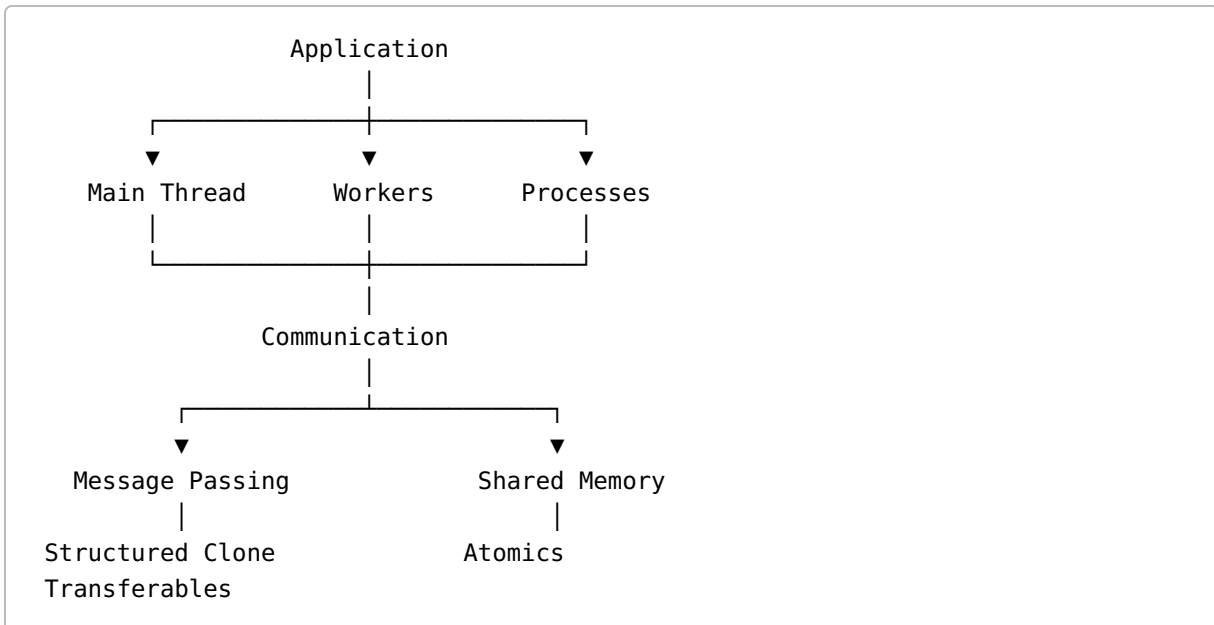
Layer 3 – Communication

- |
- ├─ Message passing
- ├─ Structured clone
- ├─ Transferables
- └─ Comlink

Layer 4 – Shared State

- |
- ├─ SharedArrayBuffer
- └─ Atomics

And above all of them:



75. Quick Revision

Worker Threads

CPU-heavy JavaScript

↓

Separate thread

child_process

Separate OS process

SharedArrayBuffer

Shared memory

Atomics

Synchronization / atomic operations

Transferable

Transfer ownership
instead of normal cloning

Worker Pool

Many jobs
↓
Limited number of reusable workers

Comlink

Worker communication
↓
RPC-like API

Web Worker

Browser background computation

Service Worker

Browser-managed background/network layer

76. Interview Questions

Fundamentals

1. What is the difference between concurrency and parallelism?
2. Is Node.js single-threaded?
3. Why do CPU-heavy operations block the Node.js event loop?
4. What are Worker Threads?
5. What is the difference between Worker Threads and `child_process`?

Memory & Communication

1. What is structured cloning?
2. What is a transferable object?
3. Why can transferring an `ArrayBuffer` be more efficient than cloning it?
4. What is `SharedArrayBuffer`?
5. What is a race condition?
6. Why do we need `Atomics`?
7. `SharedArrayBuffer` vs transferable `ArrayBuffer`?

Architecture

1. Why is creating one Worker per request often a bad idea?
2. What is a worker pool?
3. How would you handle a worker crash?
4. How would you implement job timeouts?
5. How would you implement cancellation?
6. When would you choose worker threads over child processes?
7. When would you use an external job queue instead of Worker Threads?

Browser

1. What is a Web Worker?
2. Why can't a Web Worker normally manipulate the DOM?
3. What is a Service Worker?
4. Web Worker vs Service Worker?
5. Explain the Service Worker lifecycle.
6. What is `MessageChannel`?
7. What is `MessagePort`?
8. Dedicated Worker vs Shared Worker?

Senior

1. Why doesn't adding more workers always improve performance?
2. Explain Amdahl's Law in the context of worker pools.
3. How would you design a CPU-intensive image-processing service?

4. How would you prevent a worker from monopolizing CPU?
 5. When is SharedArrayBuffer justified?
 6. Why is message passing often easier to reason about than shared memory?
 7. How would you design request/response correlation between the main thread and a worker?
 8. How would you design a fault-tolerant worker pool?
-

77. Practice Projects

Project 1 — CPU Worker

Create:

```
main.js  
worker.js
```

Send a number to the worker and calculate something CPU-intensive.

Project 2 — Worker Pool

Build:

```
Job Queue  
  ↓  
3 Workers  
  ↓  
Results
```

Requirements:

- Maximum 3 concurrent workers
 - Queue remaining jobs
 - Reuse workers
 - Handle errors
 - Handle worker exit
-

Project 3 — Transferable Benchmark

Compare:

Structured Clone
vs
ArrayBuffer Transfer

with a large binary dataset.

Measure:

Time
Memory
Throughput

Project 4 — Shared Counter

Create:

Worker 1 ┌
Worker 2 ┤ — SharedArrayBuffer
Worker 3 └

Have all workers increment a shared counter.

First implement it incorrectly:

```
shared[0]++;
```

Then fix it using:

```
Atoms.add(...)
```

Observe the difference.

Project 5 — Browser Worker

Create a web page that performs a CPU-heavy calculation.

First:

```
Main Thread
  ↓
calculation
```

Observe UI freezing.

Then:

```
Main Thread
  ↓
Web Worker
  ↓
calculation
```

Observe the UI responsiveness.

Project 6 — Service Worker Cache

Build:

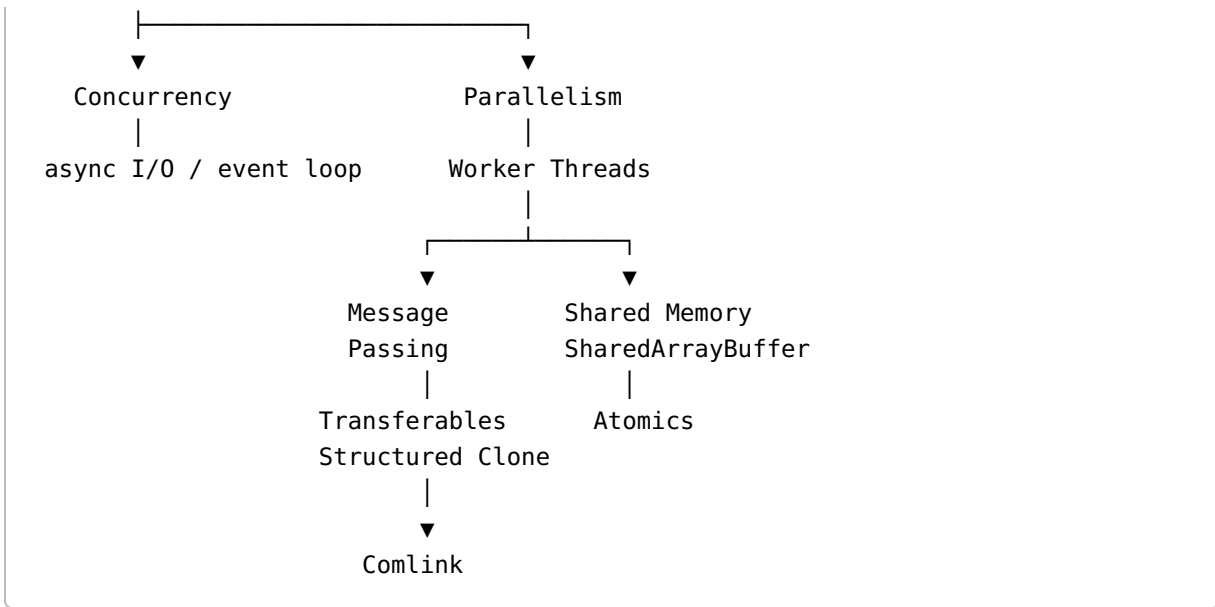
```
Browser
  ↓
Service Worker
  ↓
Cache first
  ↓
Network fallback
```

Test the application while offline.

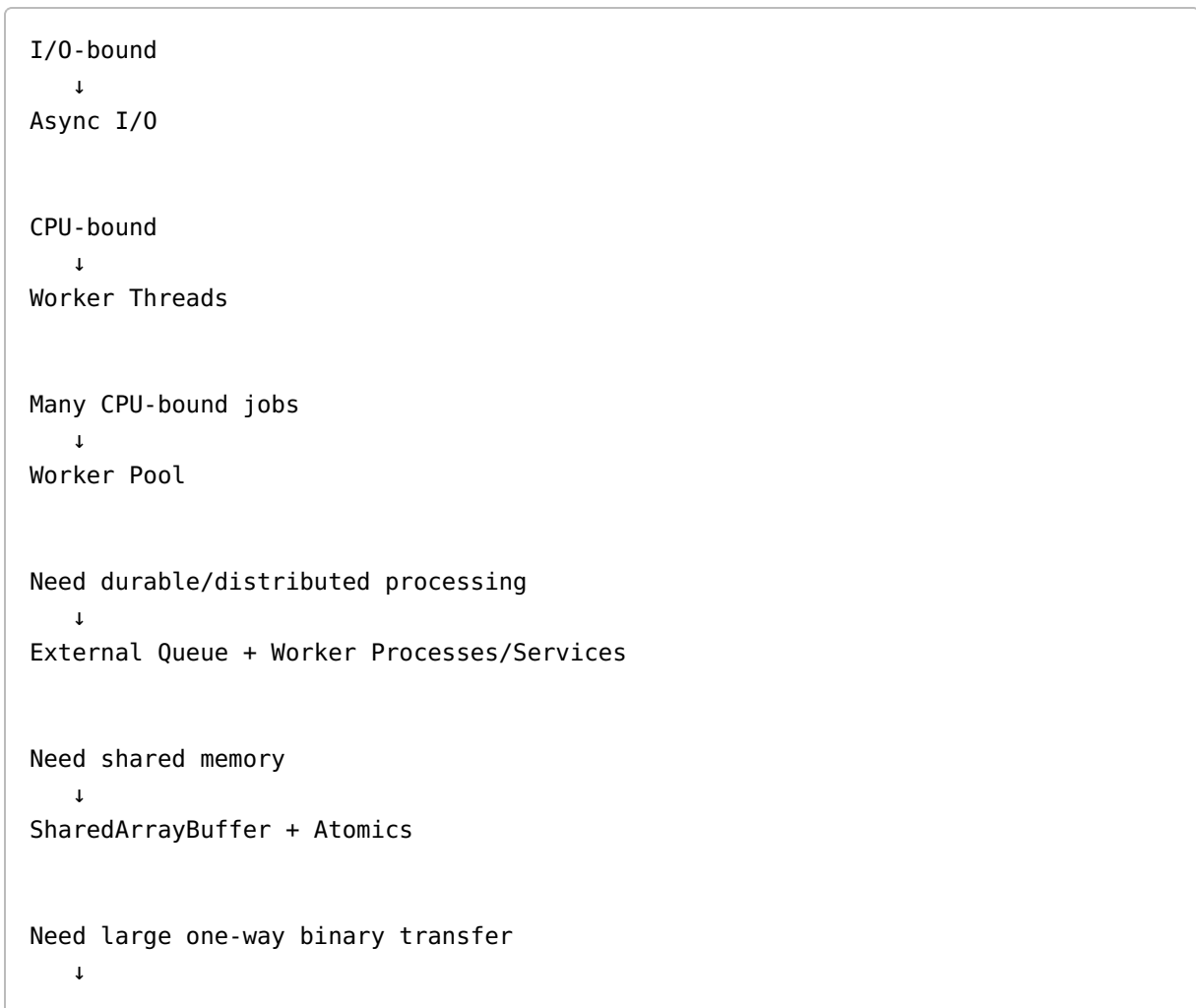
Final Mental Model

The biggest lesson from this entire group is:

```
JavaScript execution
  |
  ▼
Usually sequential on one execution thread
  |
```



For backend development, remember this decision:



Transferable objects

Browser CPU-heavy work



Web Worker

Browser network/background control



Service Worker

And the most important senior-level principle:

Concurrency is not about creating more threads. It is about controlling how work, CPU, memory, communication, and shared state interact.

A good concurrent architecture minimizes unnecessary shared state, bounds parallelism, chooses the right communication mechanism, and makes failure/cancellation explicit.